

Architecture Brief: Polyglot Architecture/Persistence and Telemetry

By: Lucas Chaparro and Narendra Nath Chalasani

Executive Summary

Apex Events is an organization that is able to process high-velocity telemetry and also uphold the integrity of financial information. Apex Events is committed to tracking what visitors are up to at the festival. This includes a variety of events such as tracking the booths that they visit, the locations where their wrist bands are scanned, and their inputs in the Apex Events app.

Polyglot Persistence Architecture was created in order to solve this issue. ACID-compliance is ensured in PostgreSQL and keeps billing secure. High-velocity telemetry data including wrist bands and booth locations, were integrated using MongoDB in the stack as relational databases are too rigid for high-volume. These two databases are able to have their combined data accessed in a unified response using the FastAPI endpoint: `Get/users/{id}dashboard`).

Dividing the Workload

Relational databases are too rigid for high-volume, variable telemetry data, therefore MongoDB was needed in order to handle and track the wrist band scans and booth visitor information and it is more suitable for higher write throughput and availability. PostgreSQL was needed in order to handle consistent data especially in important areas such as financial transactions.

	PostgreSQL	MongoDB
Primary Focus	Consistency, strict data isolation	High availability and write throughput
Data Structure	Rigid, relational tables, highly structured	More loosely structured,
Consistency Class	ACID compliant, consistency is a must	Eventual consistency
Key Operations	Handles financial data, ticket purchases, tracks seating	Track interactions with buttons in the app, wrist band scans, booths visited

PostgreSQL: ACID Compliance

Any operations that handle financial transactions and availability require precise accuracy. The architecture used with PostgreSQL ensured ACID compliance.

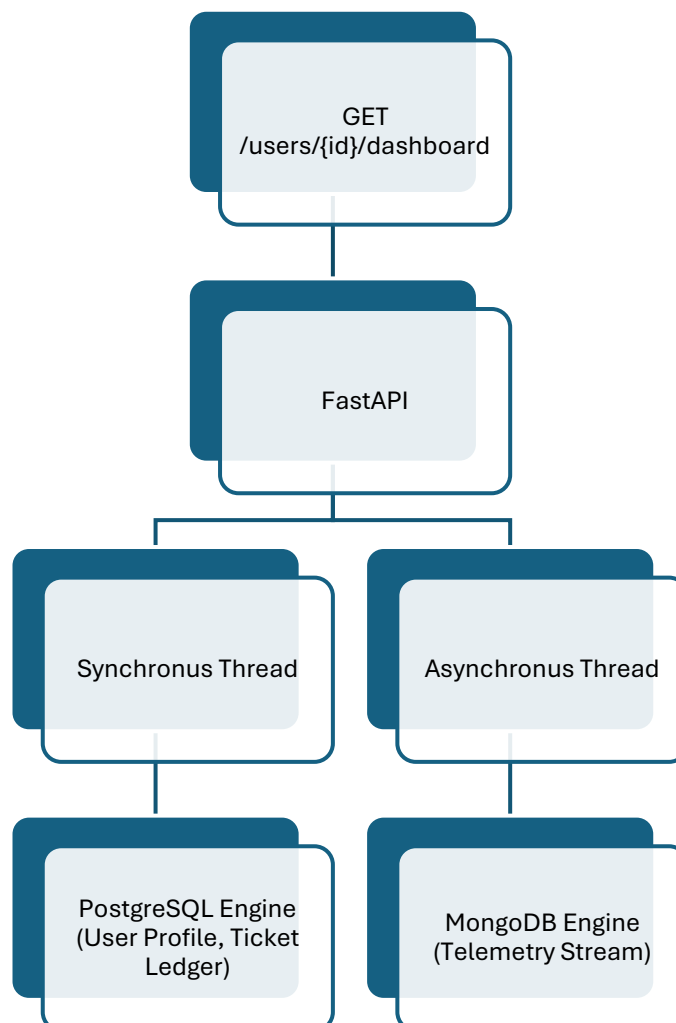
- **Atomicity:** The process for purchasing tickets involves checking the availability of seats, decreasing the availability count, and processing the financial billing information. The entire transaction will be rolled back if any step here fails.
- **Consistency:** Users are unable to duplicate purchases and cannot make orphan entries that aren't tied to anything else. There are foreign key relationships (`user_id` pointing to `users.id`) and unique indices (`unique_user_event`) in the database rules.
- **Isolation:** Row-level locks are implemented. Two users cannot modify or edit the same seat inventory at the same time, only one is allowed.
- **Durability:** Completed purchases will survive crashes and PostgreSQL can rollback to a previous state should any failures occur.

MongoDB: High-Velocity Telemetry

Operations that do not require super precise accuracy and involve large amounts of data will be handled using MongoDB. This will include handling wristband scans and knowing which ticket booths were visited. MongoDB does not include ACID compliance like PostgreSQL does, but it has a much higher throughput for reading and writing data, especially unstructured data.

Architecture Design

Both PostgreSQL and MongoDB were integrated together at the application layer using FastAPI. The following is a visual of how the architecture is set up.



When a user wants to access the dashboard, FastAPI will synchronously query indexed relational rows in users and match them with their ticket instances using PostgreSQL. FastAPI will asynchronously query telemetry data using MongoDB and does not block execution from other places. The final result is that a JSON schema will be created using the unified datasets.